HOCHSCHULE MITTWEIDA
University of Applied Sciences

BACHELORARBEIT

Herr Paul Fuchs

Die Downtime-freie Bereitstellung von Microservices in Kubernetes Umgebungen.

2026

Fakultät Angewandte Computer- und Biowissenschaften

BACHELORARBEIT

Die Downtime-freie Bereitstellung von Microservices in Kubernetes Umgebungen.

Autor:
Paul Fuchs

Seminargruppe:
MI20w2-B

Studiengang:
Medieninformatik und
Interaktives Entertainment

Erstprüfer:
Prof. Philipp Klimant

Zweitprüfer:
Dr. Beier-Grunwald

Einreichung:
Mittweida, den 27.03.2026

Bibliografische Angaben

Fuchs, Paul: Die Downtime-freie Bereitstellung von Microservices in Kubernetes Umgebungen. TODO: 5 Seiten, 14 Abbildungen, 1 Anlage mit Quelltexten, Hochschule Mittweida (FH), Fachbereich Angewandte Computer- und Biowissenschaften

Bachelorarbeit 2026

Abstract TODO

Inhaltsverzeichnis

1 Einleitung **1**

1.1 Motivation 1

2 Theoretische Grundlagen **3**

2.1 Microservices und deren Architektur 3

2.1.1 Charakteristika von Microservices 4

2.1.2 Docker 8

2.2 Kubernetes 8

2.2.1 Kubernetes Architektur 9

2.2.2 Kubernetes Komponenten 9

2.2.3 Konfiguration 10

2.2.4 Netzwerke und Services 11

2.3 Continuous Integration, Continuous Delivery, Continuous Deployment und Progressive Delivery 12

2.3.1 Deployment 12

2.3.2 Release 12

2.4 Service Mesh 13

2.4.1 Zentrale Aufgaben 13

2.4.2 Sidecar und Sidecarless Architektur 13

2.5 Persistenz in verteilten Systemen 13

2.5.1 Expand Migrate Contract - Parallel Change 14

3 Technische Strategien **15**

3.1 Rolling Updates 15

3.2 Blue-Green Deployment 16

3.3 Canary Deployment 17

4 Monitoring, Observability und Erfolg **19**

4.1 Monitoring 19

4.1.1 Leistungsanalyse 19

4.1.2 Ursachenanalyse von Fehlern 20

4.1.3 Anomalienerkennung 20

4.1.4 Anwendungsstatus 20

4.2 Parameter zur Überwachung 21

4.2.1 Logs 21

4.2.2 Metriken 21

4.2.3 Traces 21

4.3 Monitoring Ebenen 22

4.3.1 Netzwerk Ebene 22

4.3.2 System Ebene 22

4.3.3 Service Ebene 23

4.4 DORA Metriken 23

4.4.1 Erfolgsmessung der Softwarebereitstellung 24

5 Konzeption, Implementierung und Analyse **25**

5.1 Konzeption 25

5.2 Implementierung 27

A Eigenständigkeitserklärung **31**

Abbildungsverzeichnis

5.1 Systemarchitektur: Shop- und Transaction-Microservices in k3s . . 26

Tabellenverzeichnis

5.1 Datenstruktur: Shop Service (MongoDB) 26

5.2 Datenstruktur: Transaction Service (PostgreSQL) 26

Kapitel 1

Einleitung

1.1 Motivation

Kapitel 2

Theoretische Grundlagen

2.1 Microservices und deren Architektur

Microservices und Microservice Architekturen werden als Verbund mehrerer Services beschrieben, von denen jeder Service als eigener Prozess läuft und deren Kommunikation über leichtgewichtige Mechanismen, oft eine HTTP-Resource API stattfindet. Die Services werden auf Geschäftsprozesse zugeschnitten und lassen sich unabhängig voneinander bereitstellen, testen und nach der Bereitstellung skalieren. Um Microservices besser beschreiben zu können, ist ein Blick auf den historischen Bedarf sinnvoll. Microservice Architekturen sind aus der Notwendigkeit entstanden, monolithische Anwendungen aufzubrechen, um die Komplexität zu vereinfachen und neue Funktionen der Anwendung schneller bereitstellen zu können. Unternehmensanwednungen bestehen für gewöhnlich aus drei Hauptbestandteilen. Einem User-Interface für den Nutzer, einer Datenbank und der serverseitigen Anwendung, welche HTTP Requests verarbeitet, Domänenlogik ausführt, Daten aus der Datenbank abruft, aktualisiert und HTML Ansichten an den Browser sendet. Die serverseitige Anwendung wird als monolith bezeichnet, wenn die Verarbeitung aller eingehenden Requests dieser Applikation obliegt. Während dies ein erfolgreicher Weg für eine Anwendung sein kann und der natürliche Weg ist, eine neue Applikation zu entwickeln, steigt mit der Größe der Anwendung auch die Komplexität und folglich die Zeit, die benötigt wird, die Anwendung zu bauen und neue Funktionen bereitzustellen. Ein weiterer Nachteil von monolithischen gegenüber Microservice Anwendungen ist die Ressourcenverwertung. Steigen die Nutzerzahlen der Applikation, müssen mehr Anwendun-

gen bereitgestellt werden, um die Anfragen der Nutzer zu beantworten. Bei monolithischen Anwendungen wird dabei die Anzahl der serverseitigen Applikationen erhöht. Hingegen wird bei Microservice Architekturen nur die Zahl der Komponenten erhöht, die für die Verarbeitung dieser Anfragen zuständig sind. (James Lewis, Martin Fowler 2014; AWS Authors 2026)

2.1.1 Charakteristika von Microservices

Aus dem vorangegangenen Abschnitt ist bereits ersichtlich, dass es keine formale Definition von Microservices gibt. Eher wird beschrieben welche gemeinsamen Merkmale für Microservice Architekturen identifiziert werden können. Während für Definitionen mit gemeinsamen Eigenschaften gilt, dass nicht alle Microservice Architekturen, alle Charakteristika erfüllen, ist zu erwarten, dass für die meisten Architekturen auch die viele der Eigenschaften zutreffen.

Komponentisierung durch Services

Als Komponente bezeichnen James Lewis, Martin Fowler (2014) eine Einheit von Software, die individuell austauschbar und aktualisierbar ist. Weiter wird beschrieben, dass Microservice Architekturen Bibliotheken nutzen, die in ein Programm eingebunden werden und in-memory Funktionen Aufrufe ausführen. Hingegen sind Services out-of-process Komponenten, die mittels Mechanismen, wie Web-Service Requests oder Remote Procedure Calls, kommunizieren. Ein Hauptargument für den Vorzug von Services, vor Bibliotheken ist, dass jene unabhängig bereitgestellt werden können. Wenn die Anwendung aus mehreren Bibliotheken in einem einzelnen Prozess ausgeführt werden, führt eine Veränderung einer Komponente dazu, dass die gesamte Applikation erneut ausgerollt werden muss. Ist diese Anwendung jedoch in unterschiedliche Services aufgeteilt, so führen Veränderungen an einem einzelnen Service dazu, dass nur dieser neu ausgerollt werden muss, solange Entwickler keine Schnittstellen-Änderungen implementieren, die die Kommunikation zwischen den Services beeinflusst. Ziel einer guten Microservice Architektur ist es, durch kohärente Service-Grenzen und Evolutionsmechanismen, diese zu minimieren. Eine weitere Konsequenz aus der Nutzung von Services als Komponente ist eine explizitere Komponentenschnittstelle. Jedoch hat die Nutzung von Services auch Nachteile. Die Verwendung von Remote-

Aufrufen ist aufwändiger als In-Prozess Aufrufe und daher müssen Remote-API's ”grobkörniger”designed werden. Sollten zu einem späteren Zeitpunkt die Verantwortlichkeiten zwischen Komponenten wechseln, so ist dies mit einer Menge Aufwand verbunden. (James Lewis, Martin Fowler 2014)

Organisierung nach Geschäftsfähigkeiten

Nach Melvin Conway wird jede Organisation ein System entwicklen, abhängig von der vohrerrschenden Kommunikationsstruktur. Im Sinne von: ein Unternehmen besteht aus fünf Teams, daher werden 5 Services entstehen. Für eine Microservice Architektur ist der Ansatz für die Aufteilung anders. Hier sollen die Services anhand der Geschäftsprozesse aufgeteilt werden. Nach dieser Aufteilung implementieren die Services eine umfassende Softwarearchitektur für den jeweiligen Geschäftsbereich. Beinhaltend eine Benutzeroberfläche, persistenten Speicher und externen Schnittstellen. Die Teams müssen daher funktionenübergreifend aufgestellt werden und die Kompetenzen aller beinhalteten Funktionen abdecken, Nutzererfahrung, Datenbank- und Projektmanagement. (James Lewis, Martin Fowler 2014)

Produkte, keine Projekte

Einige Bemühungen Software Applikationen zu entwickeln gestaltet sich nach einem Projekt-Modell: das Ziel ist, eine Anwendung zu entwickeln, welche dann als fertiggestellt gilt. Mit der Fertigstellung wird die Applikation an ein Unternhmen übergeben, welches mit der Wartung der Anwendung betreut wird und das Entwicklerteam wird aufgelöst. Beim Ansatz der Microservice Architektur gilt jedoch eher das Prinzip ”You build it, you run it.”Bedeutend, dass ein Team ein Produkt über dessen gesamten Lebenszyklus betreut - von der Entwicklung bis zum Produktivbetrieb. Dadurch erfahren die Entwickler, wie sich die Software im Produktivbetrieb verhält und der Kontakt zu den Nutzern wird intensiviert, da sie Teile des Supports übernehmen müssen. Durch diese Mentalität entsteht eine kontinuierliche Beziehung der Entwickler zu ihrem Produkt, bei der die Entwickler dazu angehalten sind die Applikation insofern zu verbessern, dass die Nutzer eine bessere Erfahrung im Umgang mit der Anwendung erhalten. (James Lewis, Martin Fowler 2014)

Endpunkte und Nachrichtendepots

Damit Services in Microservice Architekturen miteinander kommunizieren können, werden intelligente Endpunkte und einfache Nachrichtendepots verwendet. Anwendungen, die auf Microservices basieren, zielen darauf ab, so entkoppelt und zeitgleich so kohärent wie möglich zu sein. Die Applikationen verwenden eigene Domänenlogik und fungieren als Filter. Sie empfangen eine Anfrage, wenden die entsprechende Logik an und erzeugen eine Antwort. Mithilfe einfacher Representational State Transfer Protokolle erfolgt die Kommunikation am häufigsten über HTTP-Anfrage-Antwort mittels Ressource API's und leichtgewichtiger Nachrichten. Mit der Implementierung dieser wird eine zuverlässige asynchrone Infrastruktur geschaffen, während die eigentliche Logik weiterhin in den Endpunkten der Services liegt, welche Nachrichten produzieren und verarbeiten.

Dezentralisierte Steuerung

Die Folge zentralisierter Steuerung ist, dass eine Tendenz zur Standardisierung von Technologieplattformen entsteht. Dies schränkt die Lösungsfähigkeit für unterschiedliche Probleme ein. Durch die Aufteilung der Anwendung in Komponenten kann für ein Problem das entsprechend passende Werkzeug verwendet werden.

Dezentralisiertes Datenmanagement

Die Dezentralisierung von Datenmanagement zeigt sich auf verschiedene Weisen. Auf der am meist abstrahierten Ebene bedeutet dies, dass das Konzeptmodel der Welt sich zwischen Systemen unterscheidet. Die ist ein häufiges Problem bei der Integration in großen Unternehmen. Die Vertriebssicht auf einen Kunden weicht von der Supportsicht ab. Ein von Vetrieblern bezeichneter Kunde, erscheint in der Supportebene möglicherweise gar nicht oder tritt mit jeweils unterschiedlichen Attributen auf oder mit gleichen Attributen unterschiedlicher Semantik. Ein Ansatz zur Lösung ist das Konzept des abgegrenzten Kontexts. Hierbei wird eine komplexe Domäne in abgegrenzte Kontexte unterteilt und die Beziehungen zwischen ihnen abgebildet. Dabei besteht eine natürliche Korrelation von Service- und Kontextgrenzen, die die Trennungen verdeutlicht. Während monolithische

Anwendungen für gewöhnlich eine einzige Datenbank für persistente Daten bevorzugen, gilt für Microservice Architekturen, dass jede Komponente ihre eigene Datenbank verwaltet. Dies kann durch dieselbe Datenbanktechnologie für alle Services bis hin zu völlig unterschiedlichen Datenbanksystemen reichen.

Automatisierung der Infrastruktur

Techniken der automatisierten Bereitstellung von Softwareprodukten sind gängige Praxis. Spätestens seit den DevOps Praktiken von Continuous Integration und Continuous Delivery. Die beschreiben, dass Softwareprodukte ständig weiterentwickelt und bereitgestellt werden. Dieser Ansatz wird in einem späteren Kapitel noch genauer betrachtet. Hier sei beschrieben, dass der Prozess des Bauens von Software weitgehend automatisiert abläuft. Ein einfacher Build-Prozess lässst sich mit den folgenden Schritten beschreiben. Kompilierung und Unit-Testing, Ausführen von Integrations- und Performance Tests und der anschließenden Bereitstellung für die Produktivumgebung.

Fehlertoleranz

Eine weitere Konsequenz der Nutzung von Services als Komponenten ist, dass Anwendungen so konzipiert werden müssen, dass Fehler anderer Services toleriert werden können. Jeder Service call kann aufgrund der nicht-Erreichbarkeit des ängerufenenSServices scheitern und der anfragende Service muss möglichst elegant auf den Fehler reagieren. Das ist ein klarer Nachteil gegenüber monolithischen Architekturen, da es zusätzliche Komplexität hinzuaddiert. Die Konsequenz daraus ist, dass Teams konstant evaluieren müssen, wie die Fehler der Services die Nutzererfahrung beeinflussen. Aufgrund der Fehleranfälligkeit der Services ist es wichtig schnellstmöglich Ausfälle zu erkennen und den ausgefallenen Service wiederherzustellen. Microservice Anwendungen legen hierbei großen Wert auf Echtzeitüberwachung der Services. Dabei werden sowohl architektonische Elemente, wie die Anzahl der Datenbankabfragen pro Sekunde und geschäftsrelevante Kennzahlen, wie beispielsweise die eingehenden Bestellungen pro Minute, überprüft. Diese Überwachung kann als Frühwarnsystem dienen und Entwicklungsteams veranlassen, Fehler zu erkennen und zu untersuchen.

Evolutionäres Design

Durch das Aufbrechen einer monolithischen Anwendung in Microservices entsteht bereits ein evolutionäres Design und für die Entwickler ist die Zerlegung bereits ein Werkzeug um die Kontrolle über Änderungen an der Applikation zu haben. Dies ermöglicht weiter die Anwendung zu verändern, ohne den Entwicklungsprozess zu verlangsamen. Bei der Zerlegung eines Softwaresystems in Komponenten stellt sich unweigerlich die Frage, nach welchen Prinzipien die Applikation aufgeteilt wird. Die wichtigste Eigenschaft einer Komponente ist die unabhängige Austauschbarkeit und damit Aktualisierbarkeit. Bedeutend, dass analysiert wird, wie die Komponenten aufgeteilt werden und eben jene Komponenten dann völlig neu geschrieben werden, ohne einander zu beeinträchtigen. (James Lewis, Martin Fowler 2014)

2.1.2 Docker

Docker ist der Industriestandard als Anwendungscontainerisierungsplattform.
”Container sind leichtgewichtige, eigenständige, ausführbare Softwarepakete, die alles beinhaltet um die Applikation auszuführen. Code, Laufzeitumgebung, Bibliotheken, Umgebungsvariablen und Konfigurationsdateien.”Todd R. Weiss 17-09-2024 Docker stellt zudem eine Plattform zur Speicherung von gebauten Containern. Hier werden spezifische Images hoch- und heruntergeladen.

2.2 Kubernetes

Kubernetes, auch bekannt als K8s, ist ein Open-Source-System zur Automatisierung von Deployment, Skalierung und Management von containerisierten Applikationen. Kubernetes Authors (2026a)
Kubernetes wurde 2014 von Google als Open-Source Projekt zur Verfügung gestellt und kombiniert mehr als 15 Jahre Erfahrung im Produktionsbetrieb mit Ideen und Best-Practices aus der Community. Um Anwendungen produktiv zu betreiben werden sie verpackt, dafür lassen sich Container verwenden. In einer Produktivumgebung müssen die Container, die die Anwendung betreiben, verwaltet werden und sicherstellen, dass diese keine Downtime verursachen. Hier kommt

Kubernetes ins Spiel. Kubernetes ist ein Framework das die folgenden Funktionen bereitstellt. Service Erreichbarkeit und Lastverteilung, Speicherverwaltung, automatisierte Updates für Softwareversionen, sowie Rückabwicklung der Versionen. Weiter stellt es Ressourcenbeschränkungen, Selbstheilung von Services, Passwort- und Konfigurationsmanagement, automatisierte Service-Skalierungen und die Allokation von IP-Adressen zur Verfügung. (Kubernetes Authors 2026b)

2.2.1 Kubernetes Architektur

In diesem Abschnitt werden die architektonischen Konzepte beschrieben auf denen Kubernetes aufbaut. Ein Kubernetes Cluster besteht aus einem Control-Plane und einer Anzahl an Arbeitermaschinen, sogenannten Nodes, die containerisierte Anwendungen betreiben. Auf den Worker Nodes laufen die Applikationen, während das Control-Plane die Verwaltung der Worker Nodes und der Pods im Cluster übernimmt. TODO: https://kubernetes.io/images/docs/kubernetes-cluster-architecture.svg (Kubernetes Authors 2026c)

2.2.2 Kubernetes Komponenten

Control Plane Komponenten

Die Control Plane Komponenten treffen globale Entscheidungen, die das gesamte Cluster betreffen. Für gewöhnlich laufen alle Komponenten der Control Plane auf derselben Maschine, während Container des Nutzers nicht auf dieser Maschine laufen, sondern auf anderen, dieser unterschiedlichen Maschinen.
Der *kube-apiserver* ist die Control Plane Komponente, die die Kubernetes API erreichbar macht. Der API Server ist das Frontend für die Kubernetes Control Plane. *Etcd* ist der konsistente und hochverfügbare Schlüssel-Wert Speicher für alle Kubernetes Cluster Ressourcen. Der *kube-scheduler* ist verantwortlich für die Erstellung und Zuweisung neuer Pods zu Nodes. Der *kube-controller-manager* betreibt alle Controller Prozesse im Cluster, bspw. Node, Job oder ServiceAccount Controller.

Node Komponenten

Um die Verwaltung der laufenden Pods sicherstellen zu können und die Kubernetes Laufzeit bereitzustellen, werden die folgenden Komponenten auf jeder Worker Node ausgeführt. Das *kubelet*, welches sicherstellt, dass die Container in einem Pod ausgeführt werden, die *Container Laufzeitumgebung* die sicherstellt, dass die Container auch ausgeführt werden können. Optional kann der *kube-proxy* ausgeführt werden, der Netzwerk Regeln im Cluster verwaltet und die Netzwerkkommunikation sicherstellt. (Kubernetes Authors 2026d; Kubernetes Authors 2026c)

Arbeitslast Komponenten

Die kleinste bereitstellbare Rechenkomponente eines Kubernetes Clusters ist ein *Pod*. Pods bestehen aus einem oder mehreren Containern und werden dazu verwendet eine Instanz einer Anwendung zu betreiben. Ein Kubernetes *Deployment* verwaltet eine Anzahl von Pods. Dabei wird ein gewünschter Status beschrieben, der vom Deployment Controller sichergestellt wird. Ein *ReplicaSet* wird verwendet um die gewünschte Anzahl von Pods zu jeder Zeit sicherzustellen. (Kubernetes Authors 2026e)

2.2.3 Konfiguration

ConfigMaps

Eine *ConfigMap* ist ein Objekt, welches Schlüssel-Wert Paare speichert. Diese können als Umgebungsvariablen oder Kommandozeilenargumente gelesen werden. Kubernetes Authors 2026f

Secrets

Secrets werden verwendet, um sensible Daten oder Passwörter zu speichern und für Pods verwendbar zu machen. Kubernetes Authors 2026g

Liveness, Readiness, Startup Probes

Die *Liveness Probe* bestimmt, wann ein Pod neu gestartet werden muss. Bei wiederholtem nicht-bestehen der Liveness Probe startet kubelet den Pod neu.
Die *Readiness Probe* bestimmt, wann ein Pod bereit ist, Anfragen zu bearbeiten. Dies wird bei langsamen Starts von Anwendungen verwendet oder wenn eine Abhängigkeit zu bspw. einer Datenbank besteht.
Startup Probes werden verwendet, um sicherzustellen, dass eine Anwendung innerhalb eines Pods gestartet ist. Ist diese Probe konfiguriert, werden Liveness und Readiness Proben erst gestartet, nachdem die Startup Probe erfolgreich war.

Ressourcenmanagement

Um die Rechenleistung und den Speicherbedarf von Pods zu begrenzen, können in Kubernetes Anforderungen und feste Limits gesetzt werden. Diese werden dann pro Pod vom kube-scheduler verwendet, um neue Pods auf die entsprechenden Nodes zu platzieren um der Konfiguration gerecht zu werden. Kubernetes Authors 2026h

2.2.4 Netzwerke und Services

Service

Ein Service ist eine Abstraktion der laufenden Anwendung und wird als Kommunikationspunkt von Endpunkten verwendet, sodass unter einem Endpunkt mehrere Pods erreicht werden können.

Gateway API

Die Gateway API wird verwendet um die Kommunikation zwischen Services und von Services nach außerhalb des Clusters zu konfigurieren. (Kubernetes Authors 2026i)

2.3 Continuous Integration, Continuous Delivery, Continuous Deployment und Progressive Delivery

Continuous Integration, Continuous Delivery und Continuous Deployment sind Devops Praktiken zur Sicherstellung der kontinuierlichen Einbindung und Auslieferung von Softwarefeatures und Fehlerbehebungen. (Atlassian Authors 2026) Continuous Integration beschreibt den Prozess der automatisierten Integration von Codeänderungen in das Softwaresystem. Zur Validierung der vorgenommenen Änderungen wird der Code gebaut und anschließend getestet. So kann frühzeitig auf Fehler und Integrationsprobleme reagiert werden. Continuous Integration legt seinen Fokus auf das automatisierte Testen von Codeänderungen. Continuous Delivery erweitert Continuous Integration um die Bereitstellung der neugebauten Softwareversion in einer Testumgebung, während eine Auslieferung in die Produktivumgebung noch manuell gestartet wird. Continuous Deployment ergänzt die Delivery widerum um die automatisierte Bereitstellung in der Produktivumgebung. Progressive Delivery erweitert das Continuous Deployment insofern, dass die neue Softwareversion zunächst nur einem Teil der Nutzer zugänglich gemacht und einer kontinuierlichen Erhöhung des Datenverkehrs zur neuen Version solange die Software korrekt funktioniert. Beim Fehlschlagen wird dann automatisch auf die letzte Version zurückgerollt. (Sten Pittet 2026; Atlassian Authors 2026; Argo Rollouts Authors Pittet 2026)

2.3.1 Deployment

Als Deployment definiere Ich für diese Arbeit den Prozess des Bereitstellens eines bereits gebauten Containerimages in verschiedene Entwicklungsumgebungen, bspw. Test oder Produktiv.

2.3.2 Release

Als Release definiere Ich in dieser Arbeit die Veröffentlichung einer Softwareversion für Endnutzer.
Der Unterschied von Deployment und Release ist, dass eine releaste Softwareversion Anfragen von Nutzern bearbeitet, während das Deployment den Prozess

bis zum ängesprochen”werden darstellt.

2.4 Service Mesh

Das Service Mesh ist eine dedizierte Infrastruktur Ebene, die die Kommunikation zwischen Services verwaltet. Sie besteht aus einer Komposition leichtgewichtiger Netzwerkproxies, sogenannter Sidecars. Diese sind für die zuverlässige Zustellung von Anfragen in einer komplexen Topologie von Microservices zuständig.

2.4.1 Zentrale Aufgaben

Um in der Microservice Architektur verwendbar zu sein werden vom Service spezifische Aufgaben verlangt. Das Service Mesh hilft bei der Bewältigung der folgenden Aufgabengebiete. Service Erreichbarkeit, Lastverteilung, Fehlertoleranz, Überwachung des Datenverkehrs, einer Bruchstelle für wiederkehrende Verbindungsprobleme und der Authentifizierungs- und Zugansgskontrolle.

2.4.2 Sidecar und Sidecarless Architektur

Um die Kommunikation zwischen den Services sicherzustellen verwenden Service Meshes Envoy Proxies. Beim Sidecar Pattern werden der Proxy Container in jedem Pod als zusätzlicher Prozess gestartet. Somit wird die Kommunikation in den Sidecar Container ausgelagert und eine klare Separation der Verantwortlichkeiten geschaffen. Nachteil dieses Ansatzes ist der Ressourcenoverhead, um den Proxy neben der Applikation zu betreiben. Der Sidecarless Ansatz implementiert einen Envoy Proxy pro Kubernetes Worker Node. (Chawla 2024; Hasanth und Nadig 2024)

2.5 Persistenz in verteilten Systemen

Der sich rapide verändernden Art der Bereitstellung von Software muss sich auch die Datenbankstruktur anpassen lassen. Angenommen ein Parameter wird im Code umbenannt, im Datenbankschema jedoch nicht. Dann entsteht Downtime

während des Deployments, da der neu deployte Service auf die Datenbank zugreift und eine fehlerhafte Antwort enthält. Genauso andersherum, ändert sich mit dem Release das Datenbankschema kann die aktuell stabile Version Downtime verursachen, da die alte Softwareversion noch mit dem alten Parameternamen arbeitet und fehlerhafte Antworten von der Datenbank erhält.

2.5.1 Expand Migrate Contract - Parallel Change

Um die rückwärts-kompatibilität von Software und Datenbankschemata zu gewährleisten, beschrieb Sato 2014 das Parallel Change bzw. Expand-Migrate-Contract Pattern. Während der Expand Phase wird die Schnittstelle so erweitert, dass sowohl die alte, als auch die neue Version unterstützt wird. Exisiterende Services greifen weiter auf die alte Version der Schnittstelle zu. Die Migrationsphase bezeichnet die Zeit des Deployments, während der alle Softwareversionen das neue Datenbankschema verwenden, bei Canary z.B. der Erfolg des Releases. Mit Abschluss des Deployments startet die Contract-Phase, bei der das alte Datenbankschema entfernt wird, da alle Schnittstellen das neue Datenbankschema verwenden.
Für den Anwendungsfall des Deployments wird weiter beschrieben, dass die Anwendung dieses Pattern zur Verringerung des Änderungsrisikos führt. Zusätzlich kann es dafür verwendet werden, die Deployment Orchestration verschiedener Services aufgrund unterschiedlicher Abhängigkeiten dazwischen zu entfernen. (Pramod Sadalage 05-2016; Danilo Sato 05-2014)

Kapitel 3

Technische Strategien

In Kubernetes ist das Deployment ein Werkzeug mit dem Aktualisierungen für Pods und Replicasets ermöglicht werden. Das Deployment ist ein Objekt, welches die Spezifikation für die Bereitstellung der Anwendung und damit den gewünschten Bereitstellungszustand der Applikation definiert. Zur Laufzeit stellt der Deployment-Controller sicher, dass der tatsächliche Zustand, dem gewünschten Zustand entspricht. Bei der Erstellung eines Deployment-Objektes wird ein neues Replicaset Objekt erstellt, welches für die Erstellung von Pods verantwortlich ist. Wird das Deployment gelöscht, werden auch die zugehörigen Pods und das Replicaset entfernt. Folgend werden die technischen Strategien erklärt, die verwendet werden, um die durchgängige Erreichbarkeit der Services zu gewährleisten.

3.1 Rolling Updates

Das Rolling Update ist die Kubernetes native Standard Strategie, um Pods zu aktualisieren. Bei der schrittweise die Anzahl der Pods des alten Replicaset herunterskaliert werden, während die Anzahl der Pods des neuen Replicasets erhöht werden. Dabei gibt es Kubernetes-nativ zwei mögliche Anpassungen, welche die Bereitstellung beeinflussen. *Max Surge* und *Max Unavailable* sind diese Optionen.
Max Surge beschreibt, wie viele Pods mehr als die Anzahl der gewünschten Pods während des Deployment zur Verfügung gestellt werden. Beispielsweise werden bei einem maxSurge-Wert von 50% maximal 150% der gewünschten Anzahl an Pods zur Verfügung gestellt. Max Unavailable beschreibt, wie viele Pods wäh-

rend der Aktualisierung nicht erreichbar sein dürfen. Wird dieser Wert also auf 50% gesetzt, werden initial die Hälfte aller Pods heruntergefahren, während die neuen Pods bereitgestellt werden. Es kann je nur eine der beiden Optionen auf 0 gesetzt werden. Die Vorteile dieser Strategie sind, dass kein zusätzlicher Konfigurationsaufwand bei der Implementierung dieser Strategie entsteht. Weiter ist der Verbrauch zusätzlicher Ressourcen basierend auf der *MaxSurge* überschaubar definiert. Nachteil dieser Strategie ist die Fehleranfälligkeit. Sollte die neue Version Fehler verursachen, die nicht getestet werden können, so werden diese erst in der Produktivumgebung erkannt und die Nutzererfahrung wird beeinträchtigt. Während des Deployments ist ein erhöhtes Risiko für Latenz und Error-Raten gegeben, da die Pods simultan hoch- und heruntergefahren werden. (Rakshit und Banerjee 2024; Kubernetes Authors 2026a; Agrawal 2023)

3.2 Blue-Green Deployment

Die Methode des Blue-Green Deployments ist eine downtime freie Strategie zur Bereitstellung von Microservices, die beschreibt, dass für die Applikation zwei, bestenfalls identische, Produktionsumgebungen zur Verfügung stehen. Eine Umgebung wird als blau, die andere als grün bezeichnet und zu jeder Zeit ist eine der beiden Umgebungen aktiv. Angenommen die blaue Umgebung ist aktuell aktiv und es steht das Ausrollen einer neuen Version bevor. Dann wird die grüne Umgebung mit der neuen Version konfiguriert und letzte Tests werden in dieser Umgebung ausgeführt. Sobald die Umgebung zufriedenstellend getestet wurde, wird der Router angewiesen ankommende Nutzeranfragen an die grüne Produktionsumgebung zu leiten und die blaue Umgebung ist untätig. Diese Strategie hat den Vorteil, dass ein Rollback so schnell ausgeführt wird, wie das Ausrollen der neuen Version stattfindet. Indem der Router angewiesen wird, die Anfragen wieder auf die blaue Umgebung zu leiten. Ein Nachteil dieser Strategie ist, dass zu jeder Zeit beide Umgebungen bereitgestellt sein müssen, was prinzipiell zu einer Ressourcenverdopplung führt. (Martin Fowler 2015; Agrawal 2023; Rakshit und Banerjee 2024)

3.3 Canary Deployment

Die Canary Deployment Strategie basiert auf der Blue-Green Deployment Strategie mit dem Unterschied, dass nicht die gesamte Umgebung dupliziert wird und der Datenverkehr nicht vollständig auf die eine oder die andere Umgebung geleitet wird. Beim Canary Deployment wird die neue Version parallel zur live Version ausgerollt und nur ein Teil der Nutzeranfragen werden auf die neue Version umgeleitet. Im Laufe des Deployment werden stetig mehr Nutzer auf die neue Version geleitet, um die neue Version mit produktivem Datenverkehr zu testen. Ist das Testen erfolgreich, wird die Anzahl der Applikationen der neuen Version erhöht und die Anzahl der Anwendungen der bisherigen Version werden abgeschaltet. Der Deployment Prozess dauert so lange an, bis sich entweder gegen die neue Version entschieden wird oder alle Nutzer auf die neue Version umgeleitet werden und somit die alten Versionen der Anwendung durch die neuen ersetzt wurden. Der Vorteil dieser Strategie ist, dass Ressourcen während des Deployments optimaler genutzt werden als beim Blue-Green Deployment und das die neue Version bereits vor vollständigem Ausrollen mit produktivem Datenverkehr getestet sind. Ein Nachteil dieser Strategie ist, dass derselbe Nutzer bei der ersten Anfrage auf die neue Version geleitet wird und bei der zweiten Anfrage auf die alte Version geleitet wird - und umgekehrt. Das muss, beispielsweise durch Session Affinität, verhindert werden. (Danilo Sato 2014; Agrawal 2023)

Kapitel 4

Monitoring, Observability und Erfolg

4.1 Monitoring

Der Grund eine Softwareanwendung zu überwachen ist ein robustes und resilientes Produkt zu betreiben. Dabei wird die Software von der Leistung über die Fehleranalyse bis hin zur Gesamtbewertung der Anwendung auf unterschiedlichen Ebenen geprüft. Dies hilft dabei Probleme zu identifizieren und diese schnellstmöglich zu lösen. Weiter wird die Ressourcenverwertung optimiert und nicht zuletzt wird über den Prozess hinweg eine flexible und resiliente Anwendung gebaut.

4.1.1 Leistungsanalyse

Die Analyse der Leistung der Applikation beschreibt die kontinuierliche Überwachung verschiedener Metriken, wie die Antwortzeit oder die Service-Erreichbarkeit. Dabei wird ersichtlich, wie ein Microservice auf Anfragen reagiert. Die Überwachung der Antwortzeit gibt einen Einblick darin, wie lange ein Microservice benötigt, um eine Anfrage zu bearbeiten und das Ergebnis darzustellen. Somit können Unternehmen beobachten, welche Services langsam arbeiten und eine Optimierung anstreben. Die Überwachung der Service-Erreichbarkeit stellt sicher, dass Services erreichbar sind und korrelieren mit der Nutzererfahrung, denn ein Service mit sehr hoher Erreichbarkeit zeigt dem Nutzer, dass er sich darauf verlassen kann, den Service bei Bedarf in Anspruch zu nehmen. Die Leistungsanalyse er-

möglicht es Unternehmen zu verstehen, wie sich Änderungen auf die Leistung des Softwaresystems auswirken.

4.1.2 Ursachenanalyse von Fehlern

Die Ursachenanalyse von Fehler bindet die tiefe Analyse von Fehlern, Ausnahmen und Events ein, die zu einem speziellen Fehler geführt haben. Die Hauptaufgabe dabei ist die Erkennung des zugrundeliegenden Problems, damit der Fehler nicht erneut ausgelöst wird. Dabei werden Logs verwendet, diese geben Aufschluss darüber durch welche Eventkette der Fehler hervorgerufen wurde und lassen starke Rückschlüsse auf die Ursache des Fehlers zu. Eine Technik der Darstellung dieser Eventketten ist *distributed tracing*. Nach der Identifizierung der Ursachen des Fehlers können Prioritäten dabei helfen, die kritischsten Fehler zuerst zu beheben. Das Ziel der Ursachenanalyse von Fehlern ist das Vorbeugen von wiederholtem fehlerhaften Verhalten der Applikation.

4.1.3 Anomalienerkennung

Die Anomalienerkennung beschreibt den Prozess des Findens von untypischem Verhalten des Systems oder von Nutzern und damit dem Identifizieren von Problemen der Anwendung. Diese Probleme beinhalten Sicherheitslücken oder Systemstörungen. Ansätze zur Anomalienerkennung verwenden maschinelles Lernen, statistische Methoden und grenzwert-basierte Alarmsysteme. Eine Stärke der Anomalienerkennung ist die frühe Erkennung potenzieller Angriffe auf das System.

4.1.4 Anwendungsstatus

Die Analyse des Zustands der Anwendung beginnt mit der Erreichbarkeit und endet mit der Ressourcenverwendung und der Überprüfung von Abhängigkeiten. Dies stellt einen positiven Status aller Microservices im System sicher und umfasst periodische Abfragen zur Funktionalität des Services. Die Überprüfung richtet sich dabei an Datenbanken, API's und Drittanbieter-Services. Aus der Analyse dieser Daten können Engpässe von Ressourcen festgestellt werden. Das Ziel der Über-

wachung des Anwendungsstatusses ist, die Sicherstellung der Erreichbarkeit des Softwaresystems.

4.2 Parameter zur Überwachung

Die Überwachungsparameter Logs, Metriken und Traces sind essentiell für das Verständnis der Leistung, des Verhaltens und des Zustands verteilter Systeme.

4.2.1 Logs

Logs beinhalten Textdateien, die durch Microservices generiert wurden. Sie beschreiben verschiedene Events, Error Nachrichten, Transaktionen und Veränderungen des Systemzustands. Die Analyse der Lognachrichten kann genau zeigen, wann der Fehler aufgetreten ist und welche Events zu dem Fehler geführt haben. Nachdem der Fehler aufgetreten ist, sorgen die Lognachrichten dafür, dass die Ursache für den Fehler gefunden werden kann, was die Systemzuverlässigkeit verbessert.

4.2.2 Metriken

Metriken geben einen Einblick in die Leistung und Ressourcenverwertung von Microservices. Diese umfassen, den CPU Verbrauch, Arbeitsspeicher-Nutzung, Netzwerk-Durchsatz oder traditionelle Applikationen-Layer Metriken, wie aktive Nutzer oder Transaktionen pro Sekunde. Die Überwachung dieser Metriken erlauben es Organisationen den Zustand des Systems, Leistungstrends und Anomalien zu erfassen. Diese Metriken werden kontinuierlich abgefragt und können zusammengeführt durch Grafen und Dashboards visualisiert werden, was eine proaktive Performance Analyse ermöglicht.

4.2.3 Traces

Traces erfassen detaillierte Informationen zu jeder Anfrage, die durch das System verarbeitet wird und registriert dabei jeden Microservice der an der Verarbeitung

beteiligt ist. Dies soll eine fein-granulare Ansicht der Anfragenverarbeitung ermöglichen, dabei beinhalten sie Informationen zur Latenzzeit und den Abhängigkeiten zwischen den Microservices. Mit diesem Ansatz können Unternehmen die Traces analysieren und herausfinden wo Microservices ihre Leistungen verbessern können und Verarbeitungspfade verbessert werden, um die Antwortfähigkeit der Microservices zu optimieren. Traces werden besonders wirksam in den verteilten Systemenm die heute betrieben werden, Das Wissen über die Interaktionen der einzelnen Komponenten ermöglicht die Analyse der Ursachen von aufgetrenen Problemen.

4.3 Monitoring Ebenen

Um detaillierte Einblicke in das Verhalten und die Leistung des Softwaresystems zu erlangen, ist es notwendig mehrere Ebenen zu überwachen. Diese werden folgend beschrieben.

4.3.1 Netzwerk Ebene

Die Netzwerk Ebene überwacht die Interaktionen der Services die an der Beantwortung einer Anfrage beteiligt sind und wie Daten von einem Microservice zum nächsten gesendet. Damit können potenzielle Fehlerpunkte identifiziert werden, die dazu führen, dass die Kummonikationskette bricht. Weiter kann dadurch analysiert werden, welche Datenmengen zwischen Services versendet werden. Die Latenz beschreibt, wie lange Daten brauchen um zwischen den Services ausgetauscht zu werden. Um Netzwerk Probleme, wie Package Loss oder fehlerhafte Verbindungen, zu analysieren können die überwachten Error Raten verwendet werden. Die Überwachung des Netzwerk Durchsatzes ist eine kritische Ressource um die Leistung des gesamten Systems zu bewerten.

4.3.2 System Ebene

Das Monitoring auf System Ebene fokussiert sich auch die zu grundeliegende Infrastruktur, ob physischer Server oder virtuelle Maschine. Die Leistung der Microservices hängt maßgeblich von diesen Komponenten ab. Die CPU Auslastung

liefert Einblicke darüber, wie stark die Rechenleistung genutzt wird und zeigt mögliche Ressourcenüberlastungen an. Die Überwachung von Input/Output zeigt auf welcher Rate Daten vom Datenträger gelesen und geschrieben werden, was eine direkte Auswirkung auf Microservices hat, die auf gespeicherten Daten beruhen. Die Überwachung des Arbeitsspeichers ist ebenfalls kritisch für die Leistung des Softwaresystems, da eine nicht-ausreichende Arbeitsspeicherkapazitäten nicht selten zum Ausfall des Systems führen. Das kontinuierliche Monitoring dieser Metriken auf System Ebene ist notwendig um die Leistungsfähigkeit des Softwaresystems sicherzustellen und Probleme frühzeitig zu erkennen und eine Lösung dieser zu finden.

4.3.3 Service Ebene

Auf Service Ebene werden Applikations-spezifische Metriken überwacht, die essentiell für die Leistung und das Verhalten jedes Microservices sind. Alle im System laufenden Services werden überwacht im Hinblick auf die Fehlerraten, sodass auftretende Defekte schnell erkannt und behoben werden können. Weiter ist die Überwachung der Antwortzeit ausschlaggebend, da sie zeigen, mit welcher Geschwindigkeit Anfragen verarbeitet werden und somit direkt mit der Nutzererfahrung korrelieren. Folgend ist das Monitoring von Transaktionen wichtig um zu verstehen, wie hoch die Last jedes Microservices ist und demnach dessen Fähigkeit bei mehr Anfragen weitere Last zu bearbeiten. Mit der Überwachung dieser Metriken wird die Verlässlichkeit des Softwaresystems sichergestellt.

(Faseeha u. a. 2025)

4.4 DORA Metriken

Im Zuge der Arbeit am *State of DevOps Report* entwickelte ein Google Cloud Team einen standardisierten Bewertungsmaßstab für den Einfluss der Performance von Softwarebereitstellung auf die Unternehmensleistung. Dabei wurde anhand der Lean Methode der Einfluss technischer Praktiken auf die Kultur und den Effekt der Kultur auf die Performance der Bereitstellung und der Unternehmensleistung untersucht.

4.4.1 Erfolgsmessung der Softwarebereitstellung

Forsgren, Humble und Kim 2019 Sie beschreiben vier Messgrößen der Bereitstellungsperformance: Durchlaufzeit der Bereitstellung, Deployment Frequenz, Wiederherstellungszeit der Services und die Ausfallrate bei Änderungen. Die Durchlaufzeit wird von der Zeit vom Moment der Anfrage eines Features bis zur produktiven Bereitstellung definiert. Dabei werden zwei Zeitabschnitte gesetzt. Die Zeit des Entwurfs und der Validierung des Produkts und der Zeit der Auslieferung an den Kunden. Kürzere Durchlaufzeiten bedeuten schnelleres Feedback und damit schnellere Rückmeldung zu möglichen Anpassungen. Die Deployment Frequenz misst die Häufigkeit der Bereitstellung angepasster Software in die Produktionsumgebung. Um Geschwindigkeit und Stabilität des Systems zu überprüfen wird traditionell die Zuverlässigkeit eines Systems darüber gemessen, wie lang die Zeit zwischen den Ausfällen ist. Bei konstanten Deployment-Zirkeln sind Ausfälle jedoch unvermeidbar. Demnach wird bei der Bewertung die Zeit gemessen, die es braucht Services wiederherzustellen.

Kapitel 5

Konzeption, Implementierung und Analyse

5.1 Konzeption

Konzipiert wird eine E-Commerce Applikation, die einen Shop und dazugehörige Kauftransaktionen implementiert. Es ergibt sich je ein Microservice für den Shop und für die Transaktionen. Je Microservice wird zudem eine Datenbank aufgesetzt. Für den Shop eine nicht-relationale DB und für die Transaktionen eine relationale DB. Mit der Deprecation von Ingress-Nginx stellt sich für viele Unternehmen jetzt die Frage, was die Kommunikation zwischen Services ersetzt. Kubernetes nativ bietet die Gateway API einige der Funktionen. Inwiefern jedoch ein Service Mesh nötig ist, um weiterhin downtime-freie Deployments sicherzustellen, soll im folgenden Abschnitt erörtert werden. Doch zunächst zum Aufbau. Für das Kubernetes Cluster stehen zur Auswahl: K3s oder kubeadm als zugrundeliegende Technologien. Während kubeadm von der Kubernetes Dokumentation ausgeschrieben wird, entscheide ich mich für k3s. Der Host des Clusters ist ein Raspberry Pi 4 und mittels k3d wird ein Cluster mit einer Master Node und einer Worker Node. Als Softwareprodukt habe ich mir ein e-Commerce szenario gewählt, um Datenbankmigrationen simulieren zu können, die auch Transaktionen beinhalten, welche beim Deployment auf Beendigung warten müssen, da Zahlungsvorgänge nicht doppelt abgewickelt werden sollten, UserExperience. Es wird ein Microservice für den Shop in Kotlin geschrieben, der als Serverframework kTor verwendet. Da der Shop komplexe Business Logik verwendet und sich der Komplexität

anpassen muss. Aufgrund der Variabilität der Daten wird diesem Microservice eine mongo-db zugeordnet. Für die Transaktionen, also Bezahlvorgänge wird ein Microservice in Go implementiert, der eine SQL-basierte Postgresql Datenbank verwendet. Die Datenbank Instanzen laufen auf der Master Node, während die Microservices, Prometheus und Grafana auf der Worker Node betrieben werden sollen. Zur veranschaulichung der Systemarchitektur ist folgende Abbildung:

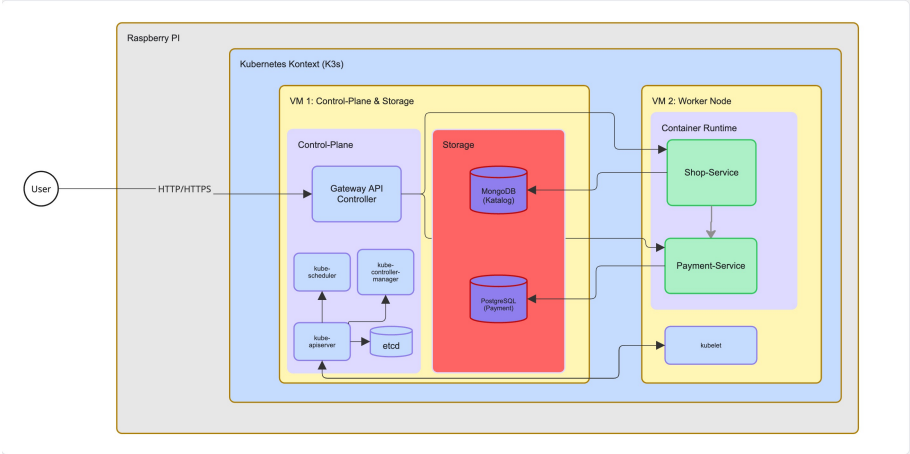

Abbildung 5.1: Systemarchitektur: Shop- und Transaction-Microservices in k3s

Als Datenstruktur habe ich ein simple Struktur verwendet.

Tabelle 5.1: Datenstruktur: Shop Service (MongoDB)

Feld	**Datentyp**	**Beschreibung**
`_id`	`ObjectId`	Identifikator des Produkts (PK)
`name`	`String`	Produktname
`description`	`String`	Produktbeschreibung
`price`	`Decimal128`	Betrag
`metadata`	`Map / Object`	Flexibles Feld
`stock_qty`	`Int32`	Lagerbestand

Tabelle 5.2: Datenstruktur: Transaction Service (PostgreSQL)

Spalte	**Datentyp**	**Beschreibung**
`id`	`UUID`	PK (random_uuid).
`order_id`	`VARCHAR(36)`	Referenz zur Bestellung
`user_id`	`VARCHAR(36)`	Identifikator des Käufers
`amount`	`NUMERIC(15,2)`	Betrag.
`currency`	`VARCHAR(3)`	Währung
`status`	`VARCHAR(36)`	Status der Bezahlung (`PENDING`, `COMPLETED`, `FAILED`)
`created_at`	`TIMESTAMP`	Zeitstempel mit Zeitzone für Audit-Logs

5.2 Implementierung

Literatur

Agrawal, Himanshu (2023). *Kubernetes Fundamentals: A Step-by-Step Development and Interview Guide*. eng. 1st ed. Berkeley, CA: Apress. ISBN: 1484297296.

Argo Rollouts Authors Pittet (2026). *Concepts*. URL: https://argo-rollouts.readthedocs.io/en/stable/concepts/ (besucht am 18. 02. 2026).

Atlassian Authors (2026). *devops*. URL: https://www.atlassian.com/de/devops (besucht am 16. 02. 2026).

AWS Authors (2026). *Microservices*. URL: https://aws.amazon.com/microservices/ (besucht am 20. 01. 2026).

Chawla, Varun (2024). „Exploring How Service Meshes Can Help To Facilitate The Development Of Microservices“. In: *2024 IEEE 15th International Conference on Software Engineering and Service Science (ICSESS)*, S. 35–43. DOI: 10.1109/ICSESS62520.2024.10719389.

Danilo Sato (2014). *Canary Release*. URL: https://martinfowler.com/bliki/CanaryRelease.html (besucht am 06. 02. 2026).

— (5-2014). *Parallel Change*. URL: https://martinfowler.com/bliki/ParallelChange.html (besucht am 20. 02. 2026).

Faseeha, Ummay u. a. (2025). „Observability in Microservices: An In-Depth Exploration of Frameworks, Challenges, and Deployment Paradigms“. In: *IEEE Access* 13, S. 72011–72039. DOI: 10.1109/ACCESS.2025.3562125.

Forsgren, Nicole, Jez Humble und Gene Kim (2019). *Das Mindset von DevOps. Accelerate*. 1. Aufl. München: Vahlen.

Hasanth, Karumuri Meher und Deepak Nadig (2024). „Evaluating the Performance of Sidecar-based and Sidecarless Cloud-native Service Mesh Solutions“. In: *2024 IEEE International Conference on Advanced Networks and Telecommunications Systems (ANTS)*, S. 1–6. DOI: 10.1109/ANTS63515.2024.10897788.

James Lewis, Martin Fowler (2014). *Microservices*. URL: https://www.martinfowler.com/articles/microservices.html (besucht am 17. 01. 2026).

Kubernetes Authors (2026a). *Kubernetes*. URL: https://kubernetes.io/ (besucht am 28. 01. 2026).

— (2026b). *Kubernetes*. URL: https://kubernetes.io/docs/concepts/overview/ (besucht am 22. 02. 2026).

— (2026c). *Kubernetes*. URL: https://kubernetes.io/docs/concepts/overview/ (besucht am 22. 02. 2026).

— (2026d). *Kubernetes*. URL: https://kubernetes.io/docs/concepts/overview/ (besucht am 22. 02. 2026).

— (2026e). *Kubernetes*. URL: https://kubernetes.io/docs/concepts/workloads/ (besucht am 22. 02. 2026).

— (2026f). *Kubernetes*. URL: https://kubernetes.io/docs/concepts/configuration/configmap/ (besucht am 23. 02. 2026).

— (2026g). *Kubernetes*. URL: https://kubernetes.io/docs/concepts/configuration/secrets/ (besucht am 23. 02. 2026).

— (2026h). *Kubernetes*. URL: https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/ (besucht am 23. 02. 2026).

— (2026i). *Kubernetes*. URL: https://kubernetes.io/docs/concepts/services-networking/ (besucht am 23. 02. 2026).

Martin Fowler (2015). *Blue Green Deployment*. URL: https://martinfowler.com/bliki/BlueGreenDeployment.html (besucht am 04. 02. 2026).

Pramod Sadalage (5-2016). *Evolutionary Database Design*. URL: https://martinfowler.com/articles/evodb.html (besucht am 18. 02. 2026).

Rakshit, Haranath und Subhasis Banerjee (2024). „Scalability Evaluation on Zero Downtime Deployment in Kubernetes Cluster“. In: *2024 IEEE Calcutta Conference (CALCON)*, S. 1–5. DOI: 10.1109/CALCON63337.2024.10914046.

Sten Pittet (2026). *devops*. URL: https://www.atlassian.com/de/continuous-delivery/principles/continuous-integration-vs-delivery-vs-deployment (besucht am 16. 02. 2026).

Todd R. Weiss (17-09-2024). *Docker for Web Developers: Getting Started with the Basics*. URL: https://www.docker.com/blog/docker-for-web-developers/ (besucht am 21. 02. 2026).

Anhang A

Eigenständigkeitserklärung

Hiermit erkläre Ich, dass die vorliegende Arbeit selbstständig und nur unter Verwendung der angegebenen Literatur und Hilfsmittel angefertigt habe. Stellen, die wörtlich oder sinngemäß aus Quellen entnommen wurden, sind als solche kenntlich gemacht. Diese Arbeit wurde in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegt.

Borna, den 16.03.2026 Paul Fuchs